Davante

15

ENTORNOS DE DESARROLLO

# Depuración de programas

# ÍNDICE

/ 1. Introducción y contextualización práctica 3

/ 2. Herramientas para la depuración de programas 4

2.1. El depurador 4

/ 3. Caso práctico 1: “IDE sin depurador” 5

/ 4. Puntos de ruptura y requisitos hardware y software 6

/ 5. Seguimiento en tiempo de ejecución y examinadores de variables 6

/ 6. Caso práctico 2: “Uso del depurador” 7

/ 7. Resumen y resolución del caso práctico de la unidad 7

/ 8. Bibliografía 8

© MEDAC 978-84-18983-00-9
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

# OBJETIVOS

*Conocer qué es un depurador.*

*Comprender las diferentes funciones de un depurador.*

*Saber los requisitos para disfrutar de un depurador.*

*Conocer el concepto de punto de ruptura.*

## / 1. Introducción y contextualización práctica

A medida que avanzamos en el desarrollo *software*, es importante tener en cuenta algunas herramientas complementarias que permiten resolver problemas de una manera más eficiente. Al aumentar el número de líneas de código, la complejidad del *software* aumenta significativamente.

Si tenemos que tratar con soluciones concurrentes, puede ser muy complicado encontrar errores en tiempo de ejecución. Por esta razón, es muy importante conocer cómo trabajan los depuradores y saber toda la nomenclatura que gira a su alrededor.

En ocasiones, interesa usar un depurador para ver realmente cómo se está comportando el código en un punto determinado sin la necesidad de que exista un error previo.

Por ejemplo, puede ser útil para ver el tipo de datos que se está almacenando en una variable, en el caso de que trabajemos con un lenguaje que no disponga de un sistema de tipos de datos intuitivo.

Es importante que cuando elijamos un entorno de desarrollo, lo adecuemos a las características de nuestro proyecto y a la previsión del tamaño de la aplicación que vayamos a desarrollar. Si elegimos un entorno en el que no tenemos depurador, podemos tener grandes problemas en el futuro para encontrar posibles errores.

Escucha el siguiente audio en el que planteamos el caso práctico que iremos resolviendo a lo largo de esta unidad.



*Fig. 1. Depurar el código puede ser muy complicado.*

*Audio intro. "Elección sin depurador"*
*https://bit.ly/3lqoKix*

# / 2. Herramientas para la depuración de programas

En muchas ocasiones, cuando escribimos el código que va a ejecutar una aplicación, es posible que tengamos errores lógicos. Desde el punto de vista del desarrollador, una vez que el programa logra compilar, comienzan las pruebas que podemos realizar.

Normalmente, cuando ejecutamos por primera vez un código que acabamos de escribir, este no realiza correctamente la tarea para la cual lo escribimos. En este momento, comienza una etapa de depuración en la que tenemos que ir refinando el código para que se vaya adaptando a nuestras necesidades. Dependiendo de la experiencia que tengamos como desarrolladores, la depuración puede durar más o menos. Para poder realizarla, podemos apoyarnos en nuestra propia experiencia o utilizar herramientas que permitan ver qué está pasando en cada momento durante la ejecución del código. Este tipo de herramientas reciben el nombre de **depuradores.**

Un depurador es una herramienta que se puede utilizar para probar y depurar programas que han sido generados con códigos necesitados de depuración. Gracias al depurador, podemos examinar el flujo de ejecución de nuestro programa y ver qué instrucciones se están ejecutando en cada momento.

Una característica que destaca del programa que depuramos es que, en realidad, no es exactamente el mismo que el que vamos a sacar a producción, pues la versión de depuración contendrá símbolos especiales para permitir controlar el flujo de ejecución. Normalmente, esta versión de depuración es un poco más lenta que la versión sin estos símbolos. Sin embargo, tenemos la ventaja de poder controlar qué hacer durante la ejecución del código.

Acudimos al depurador cuando existe un fallo en el programa que no somos capaces de averiguar sin ver qué está pasando en detalle. Nótese que algunos lenguajes de programación son más sencillos de depurar que otros.



*Fig. 2. Debemos utilizar todas las herramientas para el desarrollo.*

*Vídeo1. "Depuración después de desarrollo de código"*
*https://bit.ly/36r3Zix*

## 2.1. El depurador

Como hemos visto anteriormente, los depuradores son **herramientas que permiten controlar el flujo de ejecución de un código fuente, permitiendo a los desarrolladores saber qué está pasando en cada momento durante la ejecución.**

Cuando depuramos soluciones no concurrentes, la depuración es relativamente sencilla, sin embargo, cuando existe más de un hilo en ejecución, se puede convertir en una tarea complicada.

Para controlar el flujo de ejecución, los depuradores incorporan determinadas funcionalidades como, por ejemplo, **los puntos de ruptura** que estudiaremos a continuación. Es importante destacar que, en ocasiones, el uso del depurador puede requerir mayores requisitos *hardware*, por ejemplo, mayor uso de memoria para poder tener un programa con más código cargado en memoria para su ejecución. Es importante no subestimar la existencia de un buen depurador teniendo en cuenta diferentes perspectivas como el lenguaje utilizado para el desarrollo y el consumo de recursos *hardware* necesario. En el caso de ausencia de un buen depurador, el desarrollador se puede sentir a ciegas cuando ocurre un error que no es capaz de solucionar analizando el código. Por ello, es adecuado que cuando elijamos un entorno de desarrollo, dispongamos de este tipo de herramientas para poder aumentar nuestra productividad.

Normalmente, los depuradores pueden funcionar con un único lenguaje, pero, en otras ocasiones, existen una especie de adaptadores para poder aumentar la compatibilidad hacia múltiples lenguajes.

Otro aspecto a considerar cuando utilizamos depuradores es la protección que ofrecen sobre la memoria. De esta forma, algunos depuradores ofrecen **protección para evitar el desbordamiento**, entre otras medidas.

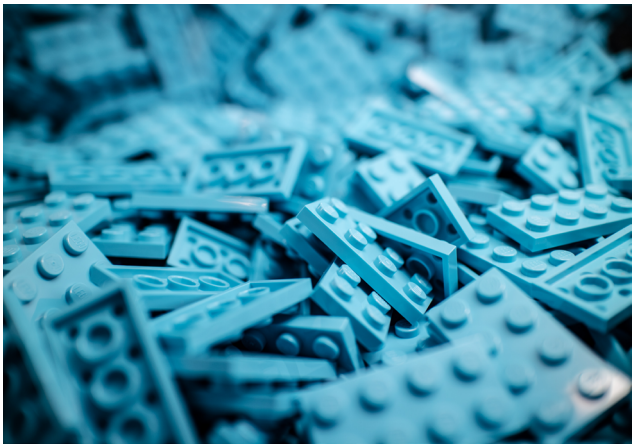


*Fig. 3. El depurador es una pieza fundamental en el desarrollo.*

*Audio 1. "Logs vs depurador"*
*https://bit.ly/3ogsZz2*

# / 3. Caso práctico 1: "IDE sin depurador"

**Planteamiento:** Nuestra empresa decide realizar el proyecto *software* en el que calculamos que vamos a tener más de 120.000 líneas de código. Dado el presupuesto que tenemos para poder comprar licencias de *software* e IDEs de desarrollo, decidimos utilizar herramientas sin la capacidad de depuración.

**Nudo:** ¿Crees que es una decisión acertada? ¿Podría afectar al desarrollo del proyecto?

**Desenlace:** La elección de herramientas de desarrollo es una de las tareas a considerar cuando comenzamos un nuevo proyecto que puede tener grandes dimensiones. Al disponer de un *software* adecuado, podemos ajustar los tiempos de desarrollo a las herramientas que tengamos a disposición. Como se nos comenta en el caso, no podemos disponer de herramientas de depuración para los posibles errores que puedan surgir. Este hecho puede afectar negativamente a la evolución del proyecto, pues no podremos depurar correctamente la aplicación y tendremos que ceñirnos a la experiencia que tenga nuestro equipo de desarrollo para poder solventar los problemas que se vayan encontrando.

En ocasiones, no podemos contar con que el equipo de desarrollo pueda afrontar esto de forma adecuadad sin saber con exactitud lo que está pasando en el código. Nótese que cuando desarrollamos un proyecto de tal envergadura, por norma, tenemos un equipo de desarrollo relativamente grande para poder afrontar cada una de las funcionalidades requeridas.

En este contexto, es muy fácil que una de esas funcionalidades no la hayamos desarrollado nosotros y la depuración lógica sea bastante tediosa.




*Fig. 4. Es necesario disponer de todas las herramientas necesarias para el desarrollo software.*

# / 4. Puntos de ruptura y requisitos hardware y software

A. **Puntos de ruptura:** En el mundo del desarrollo *software*, decimos que un **punto de ruptura** es un **lugar en el cual el *software* puede parar su ejecución quedando sin realizar ninguna tarea con el fin de que el programador pueda observar qué está pasando justo en ese momento**.

   Los puntos de ruptura se utilizan en el mundo de la depuración para permitir que los programadores puedan definir donde quieren que el *software* se detenga. Durante esta interrupción, es posible que el programador pueda inspeccionar diferentes tipos de elementos como, por ejemplo, las variables, *logs*, archivos, etc. Gracias a los puntos de ruptura, se puede tener una visión del *software* desde otro punto de vista que permite que el depurador cumpla con su función ayudando al programador. El programador tiene la potestad de poder elegir donde ubicar los puntos de ruptura, habilitarlos, deshabilitarlos, modificarlos, etc. No existe prácticamente ninguna limitación donde ubicar los puntos de ruptura, por ejemplo, podemos añadirlos en una parte condicional de un *if,* pero no en la parte del *else.*

B. **Requisitos hardware y software: Para poder llevar a cabo la depuración de programas**, es necesario cumplir con determinadas características tanto hardware como *software.*

   Desde el punto de vista del *hardware*, es necesario que la arquitectura subyacente soporte la interrupción del flujo de ejecución. En una arquitectura x86, se proveen diferentes registros con el objetivo de poder establecer puntos de ruptura que faciliten la depuración de programas. Sin embargo, existen otras arquitecturas que no incluyen esta posibilidad. En este caso, estaríamos hablando de limitaciones *hardware* para la depuración de programas.

   Desde el punto de vista del *software*, es importante que este soporte la depuración, pero requiere que esté soportado por el *hardware*, previamente.



*Vídeo 2. "Puntos de ruptura y requisitos hardware en desarrollo móvil"*
*https://bit.ly/39CXMSD*



# / 5. Seguimiento en tiempo de ejecución y examinadores de variables

A. **Seguimiento en tiempo de ejecución:** Una vez que establecemos los puntos de ruptura y lanzamos el depurador, es importante poder interactuar con este para poder decirle qué tareas realizar.

   Normalmente, los depuradores disponen de, al menos, las siguientes **funcionalidades:**

   » ***Step into.*** Permite entrar en el código de un método si tenemos un punto de ruptura fuera de él. Suele ser útil si queremos examinar el código mientras encontramos una función o procedimiento durante la ejecución.

   » ***Step over.*** A través de esa funcionalidad, podemos saltar el cuerpo de ejecución de una función/procedimiento y seguir adelante tratando la función como una única instrucción.

   » ***Step out.*** Si estamos dentro del flujo de ejecución de un procedimiento/función, podemos salir mediante esta funcionalidad.

   » ***Run.*** Con esta, podemos continuar la ejecución del programa sin ninguna interrupción, es decir, se deshabilitan el resto de los puntos de ruptura.

B. **Examinadores de variables:** Cuando se ejecuta un punto de ruptura, es decir, el software se detiene en ese punto determinado, es necesario ofrecer una determinada información al programador para que pueda ver lo que está pasando.

Esta información se puede obtener a través de los examinadores de variables y a través del flujo de código que existe en ejecución.

Por ejemplo, algunos depuradores ofrecen la pila de ejecución, en cuanto a llamadas a funciones se refiere. Además, esta información se suele añadir al contenido de las variables que se encuentran en esa porción de código. Los depuradores pueden ofrecer la información a través de los **ámbitos de visibilidad,** indicando en cada momento qué ámbito es el que está activo. Nótese que un ámbito de visibilidad es aquella parte del código que se encuentra activa en cada momento.



*Fig. 5. Los depuradores permiten examinar el contenido de las variables.*

# / 6. Caso práctico 2: "Uso del depurador"

**Planteamiento:** A la hora de desarrollar una aplicación, nos vemos forzados a utilizar un depurador, ya que no somos capaces de observar qué está pasando durante el flujo de ejecución observando únicamente los *logs*. En este contexto, vemos que necesitamos realizar una depuración del cuerpo de una función.

**Nudo:** ¿Cómo lo harías?

**Desenlace:** Como hemos visto en el tema, el depurador nos permite parar el flujo de ejecución de un programa utilizando puntos de ruptura. A través de estos puntos, podemos establecer determinados lugares que permitan observar qué sucede antes y después de ejecutar la función.

En el caso de no ser suficiente, podemos utilizar las instrucciones *step into* y *step out* para entrar y salir respectivamente del cuerpo de la función. Es importante poder realizar estas operaciones correctamente para poder obtener un resultado satisfactorio de la depuración que realicemos.

En ocasiones, es necesario generar el programa objeto de una forma determinada para poder obtener el código binario necesario que permita entrar a depurar el cuerpo de una función. Algunos depuradores permiten establecer el nivel de optimización, que consiste en reducir el código objeto eliminando información de depuración, entre otras tareas.

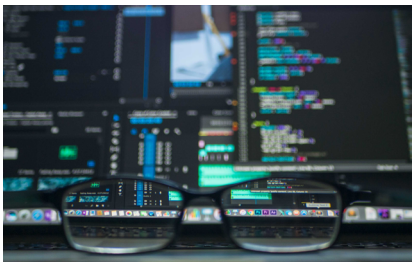


*Fig. 6. Sin el depurador, en ocasiones estaríamos a ciegas.*

# / 7. Resumen y resolución del caso práctico de la unidad

En este tema, hemos presentado el **depurador** como herramienta principal para poder conocer y parar el código durante su flujo de ejecución.

Como hemos visto, para poder ejecutar el depurador, es necesario que tanto el ***software*** como el ***hardware*** soporten dicha funcionalidad.

Hemos podido comprobar cómo la depuración se convierte en un elemento imprescindible en el desarrollo actual de aplicaciones. A medida que el número de líneas de código aumenta en una aplicación, más difícil se vuelve su mantenimiento y posible corrección de errores. En este proceso, es necesario poder ubicar los errores de una manera sencilla y rápida. En este tema, hemos visto cómo el depurador puede ayudar a esta tarea proporcionando mecanismos para encontrar rápidamente los errores y poder solucionarlo. Además, para poder utilizar el depurador, es necesario conocer el concepto de **punto de ruptura** para poder interrumpir el flujo de ejecución y adaptarnos a las necesidades que tengan los errores que vayamos encontrando a lo largo de nuestro código.

**Resolución del caso práctico inicial**

Cuando nos centramos en un proyecto de grandes dimensiones, es importante poder elegir un lenguaje que esté altamente soportado por la comunidad y, además, disponga de todas las herramientas necesarias para poder acabar el proyecto en un tiempo razonable. Como se nos comenta en el enunciado, el lenguaje que hemos elegido no dispone de un depurador, pero posee un buen sistema para introducir información durante la ejecución.

En ocasiones, podemos encontrarnos en situaciones en las que no podemos tener acceso al depurador, pero sí disponer de una información del flujo de ejecución mediante la incorporación de logs. Aunque no es la mejor elección, podría ser aceptable si introducimos el mayor número posible de estos.



*Fig. 7. Alternativas a la depuración de código.*

# / 8. Bibliografía

Shebs, S. (2018). *Debugging with GDB: the gnu source-level debugger.* Place of publication not identified: 12th Media Services

Agans, D. (2013). *Debugging: the 9 indispensable rules for finding even the most elusive software and hardware ... problems*. Place of publication not identified: Amacom.